

The goal of the project is to perform statistical analysis on a Pokemon dataset, calculating average stats by type and identifying the strongest Pokemon by type. The dataset used is a CSV file containing various details about Pokemon such as name, stats, type, etc. For the dataset, I use `serde` crate to deserialize the CSV data into Pokemon structs. The `load_data()` function handles this by reading the file with a `BufReader` and using the `csv::Reader` to deserialize each record into a Pokemon object. For data cleaning/transformation, I did not perform any of those. However, deserialization will handle any basic formatting issues if there is any. For my code structure, I have 3 modules, a `data.rs`, `analysis.rs` and `main.rs`. My `data.rs` handles loading and presentation of Pokemon data from a CSV file. As for my `analysis.rs`, it contains a function to process the data, calculating the average stats by type and identifying the strongest Pokemon by type. And lastly, my `main.rs` to load the data, run the analysis and display the results. Some key functions and types I used were `Struct` to represent the Pokemon and its attribute, `load_data()` as loading the data in the CSV, `average_stats_by_type()` function to calculate the average stats for each Pokemon type by iterating through each Pokemon. `top_pokemon_by_type()` finds the strongest pokemon and computes the total base stats and both of these functions use `HashMap`. For my cargo test, the following results are printed below. My test verifies that they are calculated correctly for the previous function on average stats and strongest pokemon and legendary type.

```
#[test]
fn test_average_stats_by_type() {
    let data = mock_data();
    let result = average_stats_by_type(&data);

    assert!(result.contains_key("Fire"));

    let fire_stats = result["Fire"];
    assert_eq!(fire_stats.0, 60);
}
```

```
Finished `test` profile [unoptimized + debuginfo] target(s) in 9.43s
Running unittests src/main.rs (target/debug/deps/finalproject-0c06d06b849d7243)
```

```
running 1 test
test test::tests::test_average_stats_by_type ... ok

test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

I created a simple test using mock data to validate the `average_stats_by_type` function. The mock data includes a single Fire-type Pokemon with specific stat values. Since there's only one Fire-type entry, the average HP is equal to that Pokemon's HP, which is 60. The test asserts that the first element of the resulting tuple (representing average HP) matches this value. This confirms that the function correctly calculates averages based on the input data.

Some of these below show the results of average stats and strongest type

Average Stats by Type:

```
{ground,rock} (3 Pokémon): HP=100, ATK=118, DEF=115
{ghost,grass} (5 Pokémon): HP=62, ATK=93, DEF=83
{flying,dragon} (2 Pokémon): HP=62, ATK=50, DEF=57
{fire,bug} (2 Pokémon): HP=75, ATK=90, DEF=55
{flying} (3 Pokémon): HP=61, ATK=76, DEF=53
{dark,normal} (1 Pokémon): HP=93, ATK=90, DEF=101
{psychic,normal} (1 Pokémon): HP=60, ATK=65, DEF=55
{ground,grass} (2 Pokémon): HP=60, ATK=55, DEF=50
{poison} (16 Pokémon): HP=66, ATK=72, DEF=67
{fire,fighting} (6 Pokémon): HP=80, ATK=100, DEF=62
{dark,dragon} (4 Pokémon): HP=109, ATK=89, DEF=65
```

Top Pokémon by Type:

```
{fairy}: xerneas with total stats 680
{ground,normal}: ursaluna with total stats 550
{fire,dark}: incineroar with total stats 530
{flying}: tornadus with total stats 580
{grass,normal}: arboliva with total stats 510
{water,steel}: empoleon with total stats 530
{ice,psychic}: mr-rime with total stats 520
{water,grass}: ludicolo with total stats 480
{psychic,fire}: victini with total stats 600
{ghost,grass}: dhelmise with total stats 517
{fire,flying}: ho-oh with total stats 680
```

Some conflicts that I had was the unusual column named in the CSV. Since type was a reserved word in Rust, ChatGPT helped me understand this and used `r#type` instead. Lastly, to have it submitted to GitHub, I also had ChatGPT to help me with the submission. I was able to successfully do so with a public key it has provided me and pasted that same ssh key to GitHub. Afterwards, I have pushed all my modules and codes onto github whilst also changing some things around like re-coding my `test.rs` as there were some errors I did not foresee, like forgetting to do `mod test` in `main.rs`, and having some errors in my `analysis.rs` for my `legendary_count` function.